
Web Based Requirements Specification & Analysis App

Product Backlog Specification

VERSIONS HISTORY

Date	Version	Description	Author
2/24/26	<1.0>	1st version of the requirements definition document	A. Zarras

1 Introduction

The objective of this project is to develop an application that facilitates the specification and analysis of the requirements of a software engineering project. In particular, the developers can specify use cases with actors, pre-conditions, flow of events, alternative flows and post-conditions. For the requirements analysis, the application provides means that allow developers to specify classes using CRC (Class-Responsibility-Collaboration) cards. A CRC card specification consists of a class name, a description of the class responsibilities and a specification of other classes with which the class collaborates. The application further allows developers to generate textual descriptions that can be used as input to tools like [PlantUML](#) and [Nomnoml](#) that generate UML use case and class diagrams.

The rest of this document is structured as follows. In Section 2 we focus on the development process that will be followed and other scoring and organizational issues. Sections 3 and 4 provide the Product Backlog, i.e., the "raw" functional and non-functional requirements that should be further analyzed to drive the design, implementation and testing of the application.

2 Development process and organization issues

To realize the project, we shall rely on a Scrum approach. Each team shall organize a number of sprints during which the team shall implement **user stories** from the project backlog and their **tests**. The deadline for the project is: 26/5/2026.

2.1 Deliverables

Definition of "done" story: A user story is done if it is **implemented correctly** and validated with one or more appropriate **tests**.

At the end of the project the Scrum team shall **deliver (via turnin)**

- **The project implementation.**
- **A Sprint report**, according to the given **Sprint report template (SprintReport-v0.doc)**, describing the sprints that they performed and the "done" user stories that have been developed during each sprint. The report should also comprise the specification of **detailed use cases, derived from the given user stories**, the **detailed design** of the application and **CRC cards** that document the responsibilities and the collaborations between the different parts of the application.
- Turn in the **project** and the other deliverables using **turnin deliverables@myy803 <your-project>.zip**, where your-project is a zip file of your Eclipse project.

2.2 Scoring

1. Working implementations of the **user stories** is **60%** of the total score.

2. **Acceptance, integration and unit tests** are **15%** of the total score.
3. **Design quality**, usage of recommended **patterns** and **best practices** to satisfy the extensibility and maintainability requirements is **15%** of the total score.
4. The quality of **reporting** is **10%** of the total score.

3 Functional Requirements / User Stories

3.1 User Management

The following user stories concern general registration and login actions.

US ID	User story
US1	As a developer, I want to create an account and log in so that I can securely access my workspace.
US2	As a developer, I want to manage my profile (name, email, password) so that I can keep my account up to date.
US3	As a user, I want to be able to log out from my account to terminate my interaction with the application.

3.2 Project Management

US ID	User story
US4	As a developer, I want to view a list of all my projects so that I can easily navigate between them.
US5	As a developer, I want to create a new project so that I can organize use cases, and CRC cards.
US6	As a developer, I want to delete a project so that I can keep my workspace tidy.

3.3 Use Cases

US ID	User story
US7	As a developer, I want to define use cases for a project by specifying actors, preconditions, main flow, and postconditions so that I can model system requirements.
US8	As a developer, I want to be able to update the use case definitions of a project so that I can refine the system requirements.
US9	As a developer, I want to view a list of all use cases for a project so that I can understand the system behavior briefly.
US10	As a developer, I want to delete a use case so that I can keep my project tidy.

3.4 CRC Cards

US ID	User story
US11	As a developer, I want to create CRC cards by specifying class names, responsibilities, and collaborations so that I can design the domain model of a project.
US12	As a developer, I want to update CRC card definitions so that I can refine the domain model of a project.
US13	As a developer, I want to link CRC cards to use cases so that I can show how project classes support the use case behavior.
US14	As a developer, I want to edit or delete CRC cards so that I can keep the domain model tidy.

3.5 Diagrams Generation

US ID	User story
US15	As a developer, I want the system to generate scripts for the visualization of the use cases of a project as use case diagrams, using Text-Based UML tools like PlantUML and Nomnoml.
US16	As a developer, I want the system to generate scripts for the visualization of the CRC cards of a project as class diagrams, using Text-Based UML tools like PlantUML and Nomnoml.

3.6 Collaboration and Versioning (optional / stretch goals)

US ID	User story
US18	As a developer, I want to share a project with teammates so that we can collaboratively edit use cases and CRC cards.
US19	As a developer, I want to comment on use cases and CRC cards so that we can discuss changes.

4 Non Functional Requirements

[NF1] Maintainability: In software engineering, maintainability is the degree of effectiveness and efficiency with which a product or system can be modified by the maintainers. In the case of this project, we specifically focus on the following concerns:

- **[NF1.1]** A first important concern is to employ an architecture that promotes low coupling and high cohesion. To this end, you can employ **Fowler's enterprise application architecture patterns** [2] that allow to clearly separate the different parts (views, controllers, domain model, database) of the application and facilitate the mapping of the domain model to the underlying database schema.

- **[NF1.2]** In the long term we want to be able to easily **extend** and **configure** the application with **different text-based UML tools, without having to change the code of the application**. The initial set of **tools** that will be supported by the application should include the following:
 - PlantUML (<https://plantuml.com/>).
 - Nomnoml (<https://www.nomnoml.com/>).

HINT To achieve **[NF1.2]** the application should be designed according to the well-known **principles** and adopt **best practices** like the **GoF design patterns**. **Specifically, you can employ the GoF strategy, template method, and parameterized factory patterns for the implementation of alternative diagram generation strategies.**

- **[NF2] Usability:** In software engineering usability concerns the ease of use and learnability. In the context of this project the application should provide a simple and user-intuitive interface. The application should also provide help, in the form of user guidelines, concerning its main functionalities of the application.

5 IDE, Java and External API

Requirements/Constraints/Recommendations

The application should be implemented in **Java**. You can use the **Spring framework**. Spring and lately **Spring Boot** are very popular technologies that facilitate the development of **Web-based applications**. As a base database management system, you can use **MySQL**. To write automated tests for the application you can use **JUnit and Mockito**. The preferred IDE is **Eclipse**.

6 References

- [1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Martin Fowler. *Catalog of Patterns of Enterprise Application Architecture*. Addison-Wesley. <https://martinfowler.com/eaCatalog/>